
Part B**Ex: 9****Title:** Metropolitan Subway System Optimization

Problem Description: A metropolitan city is planning to optimize its subway system to ensure efficient travel for its commuters. The subway system consists of various stations connected by subway lines, and each line has a different travel time. The city's transportation authority wants to find the shortest travel time between all pairs of stations.

Method: Use Dynamic programming technique to find the shortest travel time between all pairs of stations using the Floyd-Warshall algorithm

Theory Reference: Module 3

Algorithm

ALGORITHM *Floyd*($W[1..n, 1..n]$)

//Implements *Floyd*'s algorithm for the all-pairs shortest-paths problem

//Input: The weight matrix W of a graph with no negative-length cycle

//Output: The distance matrix of the shortest paths' lengths

$D \leftarrow W$ //is not necessary if W can be overwritten

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

$D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$

return D

Program

```
import java.util.Scanner;
```

```
public class Floyd {
```

```
    static void floyd(int D[][],int n)
```

```
    {
```

```
        for(int k=0;k<n;k++)
```

```
            for(int i=0;i<n;i++)
```

```
        for(int j=0;j<n;j++)

            D[i][j]=min(D[i][j],D[i][k]+D[k][j]);

    }

    static int min(int a, int b)

    {

        return(a < b ? a : b);

    }

    public static void main(String[] args)

    {

        Scanner s=new Scanner(System.in);

        int n;

        System.out.println ("Enter no. of Stations");

        n = s.nextInt ();

        int[][] cost=new int[n][n];

        //Enter the cost matrix, 0 for principle diagonol axis and no direct edge

        //(all source shortest path, source to source is zero)

        //Make 999 for no direct edge from i to j

        System.out.println ("Enter the travel time between subway lines");

        for (int i=0; i<n; i++)

        {

            for (int j=0; j<n; j++)

            {
```

```
        cost[i][j] = s.nextInt();

        if (cost[i][j] == 0 && i!=j)

            cost[i][j] = 999;

    }

}

floyd(cost,n);

System.out.println("Shortest Travel Time between all stations");

for(int i=0;i<n;i++)

{

    for(int j=0;j<n;j++)

        System.out.print(cost[i][j]+" ");

    System.out.println();

}

}

}
```

Output

```
run:
Enter no. of Stations
4
Enter the travel time between subway lines
0 0 3 0
2 0 0 0
0 7 0 1
6 0 0 0

Shortest Travel Time between all stations
0 10 3 4
2 0 5 6
7 7 0 1
6 16 9 0
BUILD SUCCESSFUL (total time: 16 seconds)
```